Análisis Comparativo de Algoritmos de Búsqueda: A* frente a Hill Climbing en un Problema de Navegación

Carlos Manuel Piñero Ramírez
Facultad de Tecnologías Libres
Universidad de las Ciencias Informáticas

6 de octubre de 2026

Resumen

Se contrastó empíricamente una búsqueda heurística global (A*) frente a una búsqueda local (Steepest-Ascent Hill Climbing) sobre un plano discreto de 15×15 celdas con obstáculos, cargado desde `dataset_coordenadas.csv`. A* resultó óptimo con un costo de 24.00 usando la distancia euclídea como heurística admisible y consistente, expandiendo 152 nodos. Hill Climbing alcanzó el mismo óptimo expandiendo solo 59 nodos, pero su éxito dependió de la topología: un barrido desde las 206 celdas libres confirmó la ausencia de óptimos locales en este tablero, mientras que un escenario adversario en forma de "U" lo dejó atrapado en un máximo local. Se concluye que la búsqueda local es eficiente pero no confiable, y que A* paga su optimalidad con un mayor número de expansiones.

1 Introducción

El problema de navegación en un espacio de estados consiste en hallar una secuencia de acciones que conduzca desde un estado inicial hasta un estado objetivo evitando regiones prohibidas. Este trabajo aborda dicho problema en un plano 2D discreto y compara dos familias de algoritmos con garantías teóricas muy distintas.

La **búsqueda heurística global** A* [1] es un algoritmo de búsqueda informada que combina el costo real acumulado $g(n)$ con una estimación $h(n)$ del costo restante. Cuando $h(n)$ es *admisible* (nunca sobreestima el costo real) A* es **óptimo**, y cuando es *consistente* es además eficiente en términos de reexpansiones. En cambio, la **búsqueda local** Hill Climbing [1] solo examina el vecindario inmediato del estado actual y avanza hacia el vecino con mejor evaluación. Es un método voraz, de bajo costo por paso, pero carente de garantías de completitud y optimalidad: puede quedar atrapado en **máximos locales**, **mesetas** o **crestas**.

El objetivo es comprender de forma empírica cómo la topología del espacio de búsqueda determina el desempeño de cada algoritmo, evidenciando los conceptos de completitud, optimalidad y óptimos locales.

2 Metodología

2.1 Definición del espacio de estados

El espacio se construyó a partir del archivo `dataset_coordenadas.csv`, que contiene 225 registros con columnas `x`, `y` y `tipo`. Cada registro define una celda del plano con uno de cuatro tipos: `libre`, `inicio`, `objetivo` u `obstaculo`. El espacio resultante es una cuadrícula de 15×15 celdas con 206 celdas libres, 17 obstáculos, el estado inicial en $(1, 1)$ y el estado objetivo en $(13, 13)$.

Formalmente, el espacio de estados es la tupla

$$M = (S,\, s_0,\, S_g,\, A,\, c),$$

donde:

- S es el conjunto de celdas transitables (libres y objetivo);
- $s_0 = (1, 1)$ es el estado inicial y $S_g = \{(13, 13)\}$ el estado objetivo;
- $A = \{\textsc{Arriba}, \textsc{Abajo}, \textsc{Izquierda}, \textsc{Derecha}\}$ define 4-conectividad, es decir, sin movimientos diagonales;
- $c(s, a, s') = d(s, s')$ es el costo de arista, igual a la distancia euclídea entre celdas consecutivas (1.0 para movimientos ortogonales).

La función de transición resultado$(s, a) = s + \Delta(a)$ es válida si el destino permanece dentro del tablero y no es un obstáculo. El espacio es finito, determinista, totalmente observable y estático.

2.2 Diseño de la función heurística

Para A* se empleó la distancia euclídea al objetivo:

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}, \tag{1}$$

con $n = (x_n, y_n)$ y el objetivo $g = (x_g, y_g)$. Esta heurística es **admisible**: en una cuadrícula con 4-conectividad cada paso cuesta al menos 1, de modo que el costo real restante es al menos la distancia Manhattan, y como $h(n) \le d_{\text{Manhattan}}(n, g)$, se tiene $h(n) \le h^*(n)$ para todo n; nunca sobreestima. Es además **consistente**, pues para toda arista (n, n') de costo $c(n, n') = 1$,

$$h(n) \le 1 + h(n') \quad \Longleftrightarrow \quad h(n) - h(n') \le d(n, n') = 1, \tag{2}$$

desigualdad que se cumple por la desigualdad triangular de la distancia euclídea. La admisibilidad garantiza la **optimalidad** de A* y la consistencia evita reexpansiones.

Para Hill Climbing se usó como función de evaluación el negativo de la misma heurística,

$$f(n) = -\,h(n), \tag{3}$$

de modo que maximizar $f(n)$ equivale a minimizar la distancia al objetivo. El algoritmo solo evalúa los vecinos inmediatos del estado actual (*steepest ascent*).

2.3 Algoritmos implementados

A* se implementó con una cola de prioridad (`heapq`) ordenada por $f(n) = g(n) + h(n)$, con conjuntos de abiertos y cerrados y un diccionario de predecesores para reconstruir la ruta. Hill Climbing se implementó en su variante *steepest ascent* y, para mitigar los óptimos locales, se le añadió **reinicios aleatorios**: al estancarse, reinicia desde una celda libre aleatoria conservando la mejor trayectoria. Ambos fueron implementados desde cero, usando exclusivamente `numpy`, `pandas`, `matplotlib` y `heapq`. El código completo se incluye en el Anexo A y en el notebook `analisis_busquedas.ipynb`.

3 Resultados Experimentales

La Tabla 1 resume las métricas. A* se ejecutó una vez (es determinista); Hill Climbing se ejecutó 30 veces con semillas distintas, por ser estocástico. Ambos resolvieron el problema con el mismo costo óptimo de 24.00 (25 nodos), pero con un número de expansiones muy diferente.

Cuadro 1: Comparación de métricas de desempeño sobre el mismo dataset.

Métrica	**A***	**Hill Climbing**
¿Llegó al objetivo?	Sí	Sí (100% de 30 corridas)
Longitud del camino (nodos)	25	25.0 ± 0.0
Costo total (distancia euclídea)	24.00	24.00 ± 0.00
Nodos expandidos	152	59.0 ± 0.0
Tiempo de ejecución (s)	0.001713	0.000575
Reinicios aleatorios	—	0.0

La Figura 1 muestra el espacio de estados con los obstáculos y los puntos de inicio y objetivo. La Figura 2 presenta la ruta óptima de A* junto con el orden de expansión de los nodos. La Figura 3 compara la ruta de A* con la mejor ruta hallada por Hill Climbing: nótese que, aunque son distintas, ambas tienen costo 24.00.



Figura 1: Espacio de estados cargado desde `dataset_coordenadas.csv`.

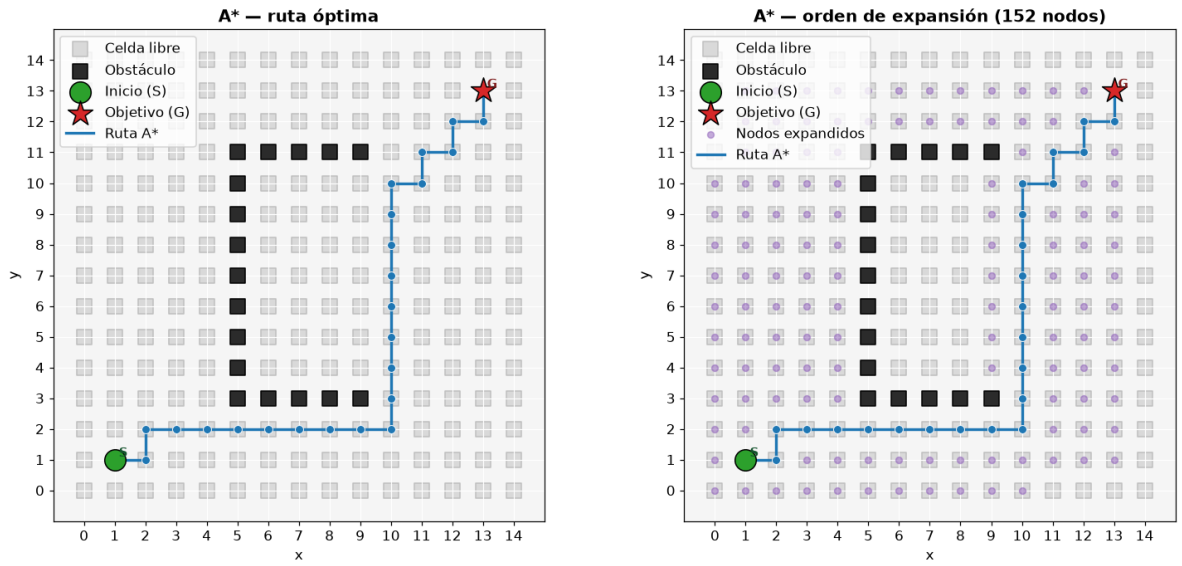


Figura 2: A*: ruta óptima (izquierda) y orden de expansión de los 152 nodos (derecha).

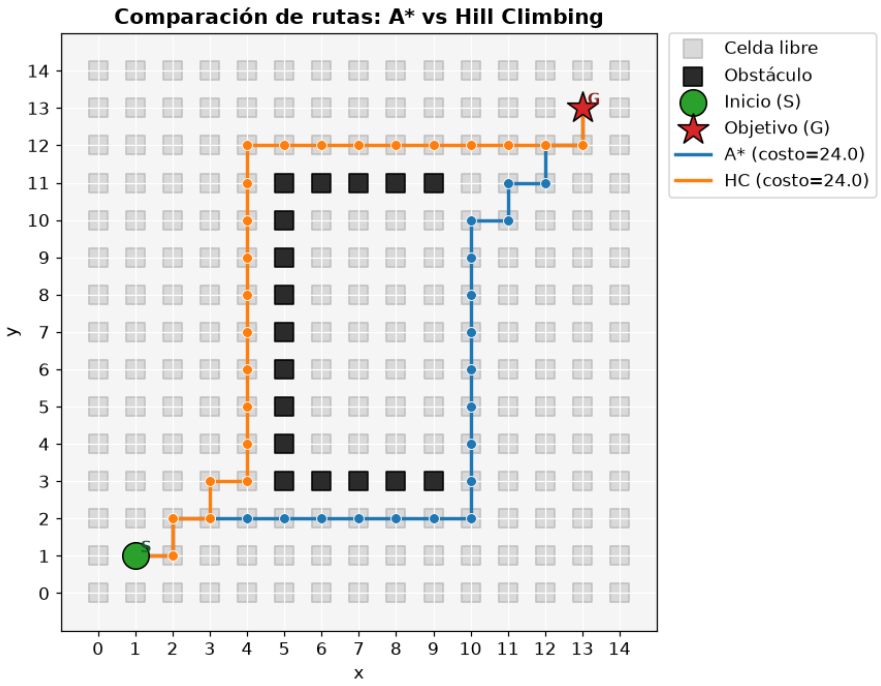


Figura 3: Superposición de la ruta de A* y la mejor ruta de Hill Climbing.

La Figura 4 analiza el costo computacional y la naturaleza estocástica de la búsqueda local. La Figura 5 presenta el escenario adversario descrito en la Sección 4.

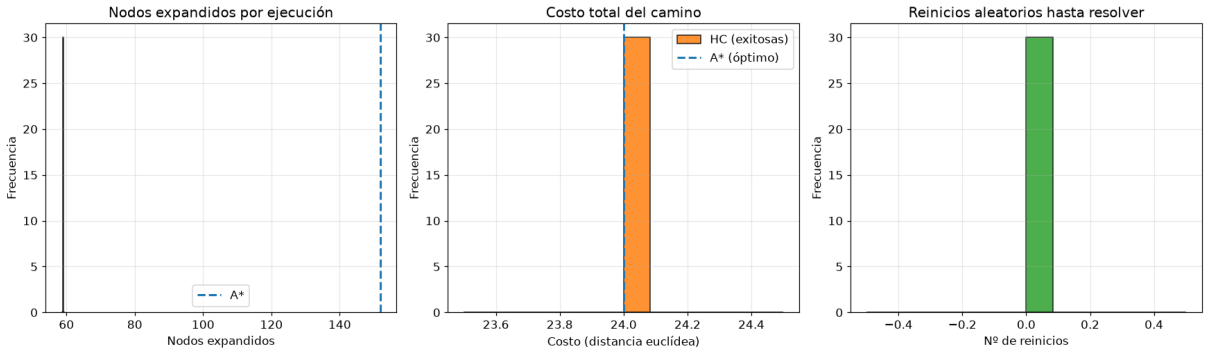


Figura 4: Nodos expandidos, costo del camino y reinicios aleatorios de Hill Climbing.

4 Análisis y Discusión

4.1 Óptimos locales, mesetas y la topología del espacio

Contrario a la intuición inicial, en *este* dataset Hill Climbing no falló: alcanzó el óptimo global en las 30 corridas y expandió menos nodos que A* (59 frente a 152). Para explicarlo es necesario analizar la topología. El estado objetivo se encuentra en la esquina inferior derecha, $(13, 13)$, y el muro de obstáculos situado en $x = 5$ es **convexo respecto al objetivo**: en todo el espacio transitable, moverse hacia abajo o hacia la derecha no aleja al agente de la meta, y los obstáculos *nunca* lo obligan a retroceder. En consecuencia, la función de evaluación $f(n) = -h(n)$ decrece de forma monótona y la trayectoria voraz coincide con una ruta óptima.

Para verificar que esto es una propiedad del tablero y no una casualidad, se ejecutó Hill Climbing voraz **sin reinicios desde las** 206 **celdas libres**: en el 100% de los casos se alcanzó el objetivo. Es decir, el paisaje de evaluación de este problema **no posee máximos locales, mesetas ni crestas** relevantes. Este resultado refuerza una idea central: el desempeño de la búsqueda local depende por completo de la forma del espacio, no de una propiedad intrínseca del algoritmo.

Para evidenciar la debilidad estructural de Hill Climbing se diseñó un **escenario adversario controlado** (Figura 5): una concavidad en forma de "U" que abre en dirección *opuesta* al objetivo. La evaluación $f(n) = -h(n)$ conduce al agente hacia el fondo de la "U", donde todos los vecinos empeoran; Hill Climbing se detiene tras solo dos pasos en el estado $(5, 8)$, a una distancia de 7.00 de la meta: un **máximo local**. A* bordea el obstáculo y encuentra la solución con costo 14.00. La región plana del fondo de la "U" corresponde además a una **meseta**: varios estados comparten el mismo valor de evaluación, lo que en ausencia de movimientos laterales provoca el estancamiento.

4.2 Optimalidad y costo computacional de A*

A* garantizó la optimalidad *por construcción*: la heurística de la Ecuación (1) es admisible (Ecuación (2)), por lo que A* expande el objetivo solo cuando su costo g es óptimo. Esta

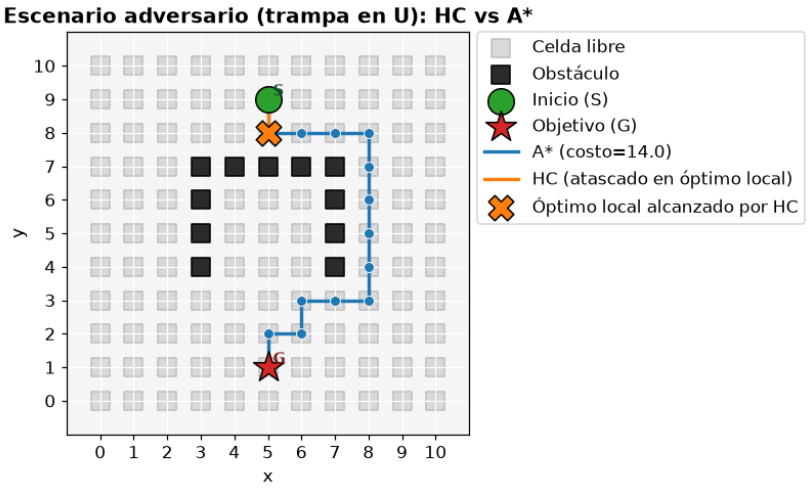


Figura 5: Escenario adversario en "U": HC queda atrapado en un óptimo local mientras A* bordea el obstáculo.

garantía es independiente de la topología del tablero: incluso en el escenario adversario, donde Hill Climbing fracasa, A* halló la ruta óptima.

El precio de la garantía es computacional. A* expandió 152 nodos frente a los 59 de Hill Climbing, un factor de aproximadamente 2.6×. La diferencia se debe a que A* mantiene una **frontera** (cola de prioridad) que puede crecer de forma exponencial, mientras que Hill Climbing solo conserva el estado actual y su vecindario. En términos de complejidad, A* es $O(b^d)$ en tiempo y espacio en el peor caso, frente al carácter de paso constante de la búsqueda local. Asimismo, A* requiere que $h(n)$ sea admisible; una heurística mal diseñada destruiría la optimalidad.

Cabe señalar que en este tablero la heurística euclídea es poco discriminante (hay muchos empates de distancia), lo que obliga a A* a expandir más nodos de los que expandiría una heurística más informada. Esto explica por qué, aun siendo óptimo, A* resultó más "costoso" que la búsqueda local en este caso concreto.

4.3 Ćuando preferir búsqueda local sobre A*

Hill Climbing resulta preferible cuando el espacio es enorme, la función de evaluación es suave (sin óptimos locales significativos) y basta una solución *suficientemente buena*. Ejemplos típicos incluyen la optimización de parámetros continuos, el ajuste de hiperparámetros, la asignación de recursos en tiempo real y la planificación con presupuestos de cómputo estrictos. En cambio, A* se justifica cuando la **optimalidad** o la **completitud** son requisitos críticos: navegación de robots, planificación de rutas, videojuegos con garantías de camino mínimo y, en general, cualquier escenario donde fallar o encontrar una ruta subóptima tenga consecuencias.

5 Conclusiones

La comparación empírica permitió constatar que A* es óptimo y completo gracias a la admisibilidad y consistencia de la distancia euclídea, a costa de expandir más nodos y mantener una frontera en memoria. Hill Climbing, en cambio, resultó sorprendentemente eficiente en este tablero (mismo costo óptimo con menos de la mitad de expansiones),

pero esa eficiencia dependió de una topología benigna: el barrido desde las 206 celdas libres confirmó la ausencia de óptimos locales. El escenario adversario en "U" demostró la fragilidad estructural del método, que quedó atrapado en un máximo local donde A* triunfó sin dificultad.

La lección principal es que la idoneidad de un algoritmo de búsqueda no puede juzgarse en aislamiento, sino en función de la **topología del espacio** y de los **requisitos de garantía**: A* cuando la optimalidad es obligatoria; búsqueda local cuando la eficiencia prima y el paisaje de evaluación es favorable. Los reinicios aleatorios mejoran la robustez de Hill Climbing pero incrementan el número de evaluaciones y no restauran ninguna garantía teórica.

Referencias

[1] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson, 2020.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael. *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics, 4(2):100–107, 1968.

A Código principal

```
def distancia_euclidiana(a, b):
    return float(np.hypot(a[0] - b[0], a[1] - b[1]))

MOVIMIENTOS = [(1, 0), (-1, 0), (0, 1), (0, -1)]

def obtener_vecinos(nodo, datos):
    vecinos = []
    for dx, dy in MOVIMIENTOS:
        candidato = (nodo[0] + dx, nodo[1] + dy)
        if candidato in datos["libres"] or candidato == datos["objetivo"]:
            vecinos.append(candidato)
    return vecinos

def costo_camino(camino):
    return sum(distancia_euclidiana(camino[i], camino[i + 1])
               for i in range(len(camino) - 1))

def a_estrella(datos):
    inicio, objetivo = datos["inicio"], datos["objetivo"]
    contador = itertools.count()
    abiertos = [(distancia_euclidiana(inicio, objetivo), next(contador), inicio)]
    g, padres, cerrados = {inicio: 0.0}, {inicio: None}, set()
    while abiertos:
        _, _, actual = heapq.heappop(abiertos)
        if actual in cerrados:
            continue
```

```
        cerrados.add(actual)
        if actual == objetivo:
            break
        for vecino in obtener_vecinos(actual, datos):
            if vecino in cerrados:
                continue
            nuevo_g = g[actual] + distancia_euclidiana(actual, vecino)
            if vecino not in g or nuevo_g < g[vecino]:
                g[vecino] = nuevo_g
                padres[vecino] = actual
                f = nuevo_g + distancia_euclidiana(vecino, objetivo)
                heapq.heappush(abiertos, (f, next(contador), vecino))
    # ... reconstruccion del camino mediante 'padres' ...

def hill_climbing(datos, max_reinicios=50, max_iter=500, semilla=0):
    rng = random.Random(semilla)
    inicio, objetivo = datos["inicio"], datos["objetivo"]
    libres = list(datos["libres"])
    for reintento in range(max_reinicios):
        actual = inicio if reintento == 0 else rng.choice(libres)
        visitados = {actual}
        for _ in range(max_iter):
            if actual == objetivo:
                return True, visitados
            vecinos = [v for v in obtener_vecinos(actual, datos)
                       if v not in visitados]
            if not vecinos:
                break
            siguiente = min(vecinos,
                            key=lambda v: distancia_euclidiana(v, objetivo))
            if distancia_euclidiana(siguiente, objetivo) < \
                    distancia_euclidiana(actual, objetivo):
                actual = siguiente
                visitados.add(actual)
            else:
                break  # optimo local / meseta
    return False, visitados
```